efrei
PARIS PANTHÉON-ASSAS UNIVERSITÉ

Java1: Fondamentaux de la POO

- Notion de classe
- Instance (Objet)
- Attributs
- Constructeurs
- Constructeur de copie
- Le mot clé **this**
- Méthodes

Coordination : Cherifa BEN KHELIL
cherifa.ben-khelil@efrei.fr

Support préparé par : Asma Gabis asma.gabis@efrei.fr

01

La Programmation Orientée Objet

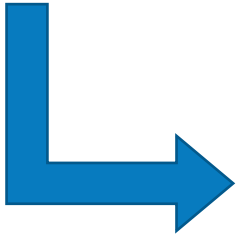
01

Programmation Orientée Objet

Besoin

Nous avons souvent besoin de regrouper, **dans une même entité**, différentes propriétés et/ou informations d'un même "**objet**".

- Un étudiant : nom, prénom, numéro de carte d'étudiant, adresse , notes
- Un joueur vidéo : pseudo, score, nombre de vies
- Une date : jour, mois , année
- Un article (dans un magasin) : code, libellé, prix unitaire, quantité en stock



- Est-il possible d'utiliser un tableau pour regrouper les différentes propriétés ?
- Comment faire pour représenter plusieurs **objets** ? Avec plusieurs tableaux ?!!!!

Programmation Orientée Objet

Objectifs

- La **P**rogrammation **O**rientée **O**bjet (**POO**) permet de créer des nouveaux types à travers la notion de classe.

- La **P**rogrammation **O**rientée **O**bjet va nous donner un certain nombre d'outillages permettant davantage de **Robustesse**, de **Modularité**, de **Lisibilité** à vos programmes, ce qui va dans le sens d'une meilleure **Maintenabilité**.
  - **Robustesse** par rapport au changement: Si votre programme est amené à changer un jour, à être étendu ou qu'on ne veut pas être dans l'obligation de tout réécrire
  - **Robustesse** face aux erreurs de manipulation, par exemple des données.

Programmation Orientée Objet

Concepts de base

- La programmation orientée objet repose sur quatre concepts centraux :
  - **Encapsulation,**
  - **Abstraction,**
  - **Héritage** et
  - **Polymorphisme**,

qui permettent de mieux organiser les programmes dans le sens de la **robustesse, lisibilité, modularité et maintenabilité**

- Ces concepts centraux ne sont pas spécifiques à un langage, il s'agit des concepts centraux de l'orienté objet.

Programmation Orientée Objet

Notion de classe

Une classe est l'abstraction d'un ensemble d'objets qui possèdent une structure identique contenant :

- Liste des attributs : représentant les caractéristiques ou propriétés de la classe
- Liste des méthodes : représentant des comportements ou des opérations

Créer une classe = Créer un nouveau type

Voiture

Propriétés/ Attributs / Données

- Couleur
- Nombre de portes
- Motorisation
- Immatriculation
- …

Équivalent aux attributs d'une structure en langage **C**

Comportement / méthodes / Traitements

- Avancer
- Reculer
- Changer de vitesse
- Ouvrir une porte
- …

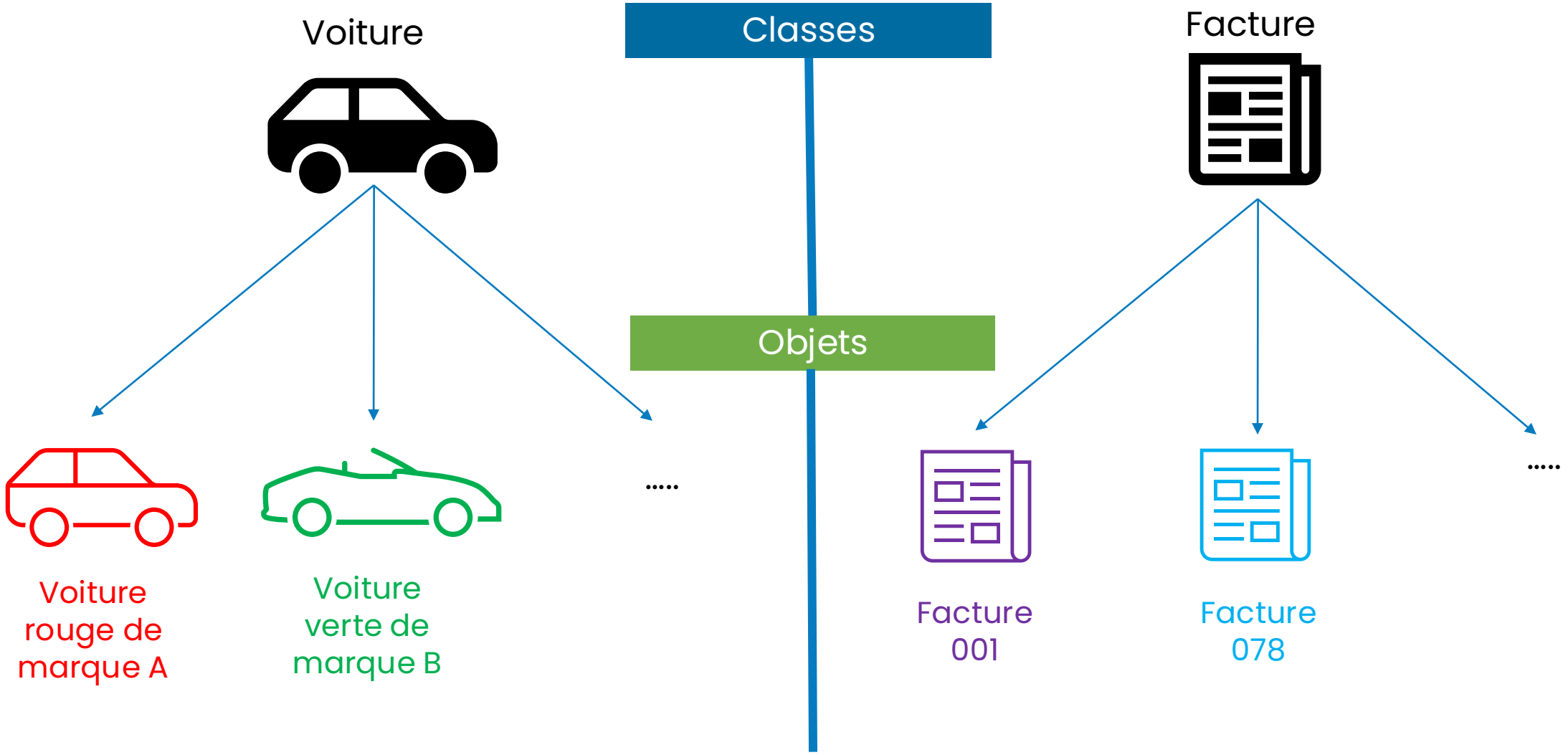
Programmation Orientée Objet

Notion d'objet

- **Un objet** est une variable **déclarés dans un type classe** (type non primitif)
- Dans la POO, l'on appelle **un objet** : une instance d'une classe.
- Les concepts de classe et d'objet **sont indépendants**

Programmation Orientée Objet

Notion d'objet



Classe et instance, type et variable

- Une classe est :
  - ✓ Une représentation abstraite d'un ensemble d'objets
  - ✓ Une description qui porte autant sur les données que sur les fonctions qui leurs sont appliquées
- La réalisation particulière d’une classe est une instance.
- Une instance = un objet

02

La POO en Java

02

La POO en JAVA

La classe

- En java une classe se déclare par le mot **class**

Mot clé du langage : marquant la création d'une nouvelle classe

class NomClasse

Nom de la classe : commence toujours par une **majuscule**

{

// Les éléments de la classe

}

Accolades début / fin délimitant le contenu d'une classe

Exemple

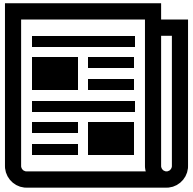
```
class Rectangle
{
  …
}
```

La POO en JAVA

Où déclarer les classes ?

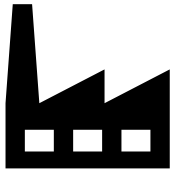
- **Chaque** classe **DOIT** être déclarée dans un fichier **propre à elle**
- Le programme principal (la méthode *main)* est lui aussi dans une classe spéciale exécutable se trouvant **dans un fichier différent des fichiers des autres classes**.

Voiture
Voiture.java



Facture
Facture.java

Etudiant
Etudiant.java



Usine
Usine.java

La POO en JAVA

Structure générale d'un programme Java

```
public class BankAccount
{

      .....

}
```

BankAccount.java

```
public class Client
{

      .....

}
```

Client.java

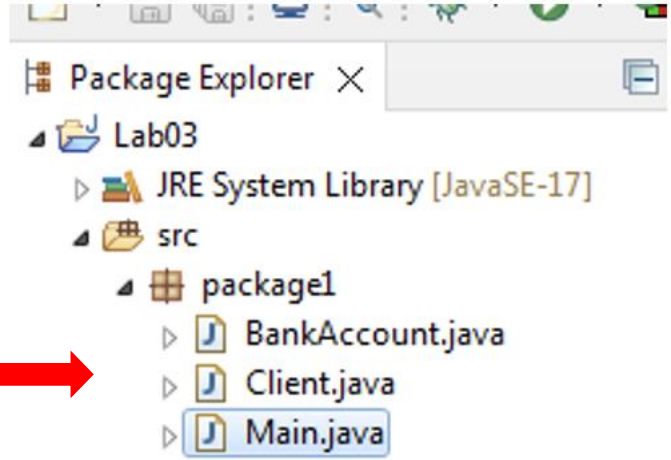
```
public class Main {

      public static void main(String[] args)
      {
            ....
      }

}
```

Main.java

Explorateur de projets dans un IDE

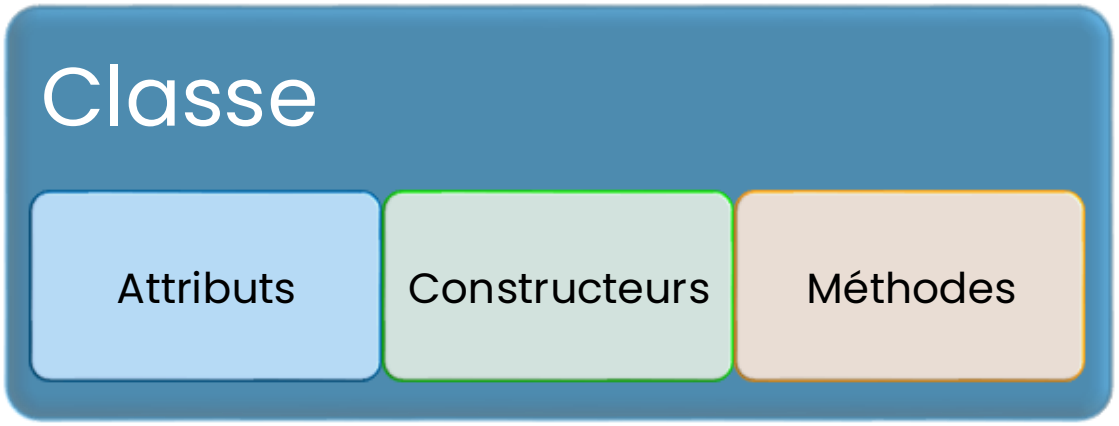


La POO en JAVA

Les éléments d'une classe

Dans une classe Java, nous définissons 3 éléments essentiels :

- **Les attributs** : représentant les propriétés de la classe
- **Un ou plusieurs constructeurs** : pour l'instanciation d'objets
- **Les méthodes** : représentant les comportements ou les actions



La POO en JAVA

Les attributs

- Les attributs sont des variables qui représentent l'état interne d'un objet.

- La syntaxe de la déclaration des attributs est la suivante:

Type nomAttribut ;

Exemple :

Les attributs *hauteur* et largeur de type double de la classe Rectangle pourront être déclarés par :

```
class Rectangle {
        double largeur;
        double hauteur;
        //...
    }
```

La POO en JAVA

Le constructeur

Un constructeur est une méthode particulière:

- Elle n'a pas de type de retour
- Invoquée systématiquement lors de l'instanciation (création) d'un objet
- Permet **d'allouer la mémoire nécessaire** à l'objet s'il **ne porte pas de paramètres d'entrée**
- Permet **d'allouer la mémoire** et **d'initialiser les attributs** de l'objet s'il **porte des paramètres d'entrée**.

Syntaxe de base:

```
class NomClasse
{
    // Liste des attributs
    NomClasse (liste-paramètres)
    {
      // Corps du constructeur
    }
}
```

Le constructeur porte EXACTEMENT le même nom que sa classe

La POO en JAVA

Instanciation d'un objet

- Une variable de type classe est instanciée et est appelée : **objet**
- L'instanciation d'un objet permet sa création en passant par les étapes suivantes :
  - ✓ Le constructeur est invoqué
  - ✓ L'espace mémoire est créé

Syntaxe de base:

Mot clé du langage : OBLIGATOIRE !!

NomClasse nomObjet = new Constructeur (arguments) ;

Type de l'objet

Nom de la variable objet choisi par le développeur. Toujours choisir des noms significatifs

Nom du constructeur = nom de la classe

A mettre en fonction du constructeur appelé

La POO en JAVA

Instanciation d'un objet

Exemple

Point.java

```java
public class Point
{
    // Liste des attributs
    int x;
    int y;

    // Constructeur
    Point()
    {

    }
}
```

Main.java

```java
public class Main
{

    public static void main(String[] args)
    {
        // Instanciation d'un objet Point
        Point p0 = new Point();

    }
}
```

Comment faire pour lire / modifier les attributs de l'objet p0 à partir du main ?

Accès à un attribut

1. À partir de sa classe

Si une méthode d'une classe souhaite manipuler un ou plusieurs attributs de la même classe, il lui suffit d'utiliser le nom de l'attribut comme une variable classique.

Point.java

```
public class Point
{
    // Liste des attributs
    int x;
    int y;

    // Constructeur
    Point()
    {
        x = 0;
        y = 3;
    }
}
```

Accès à un attribut

2. À partir d'un programme externe

Si le programme souhaitant accéder à un attribut d'une classe **C** donnée appartient à une autre classe (ex : programme *main*) :

- Instancier un objet de la classe **C**
- Accéder à l'attribut en question à travers l'objet créé en utilisant le (**.**)

nomObjet.attribut

La POO en JAVA

Accès à un attribut

2. À partir d'un programme externe

Point.java

```java
public class Point
{
    // Liste des attributs
    int x;
    int y;

    // Constructeur
    Point()
    {

    }
}
```

Main.java

```java
public class Main
{

    public static void main(String[] args)
    {
        // Instanciation d'un objet Point
        Point p0 = new Point();
        // Modification de la valeur de x
        p0.x = 0;
        // Affichage de la valeur de y
        System.out.println(p0.y);

    }
}
```

À votre avis quelle est la valeur qui sera affichée pour l'attribut y de l'objet p0 ?

Les valeurs par défaut des attributs

Lorsqu'un attribut est **déclaré mais non initialisé**, les valeurs par défaut qu'il contient dépendent de son type.

Type	Valeur par défaut
int	0
double	0.0
boolean	false
Objet (classe)	null

La POO en JAVA

Les valeurs par défaut des attributs

Exemple

Test.java

```java
public class Test
{
    // Liste des attributs
    int x;
    double y;
    boolean b;

    // Constructeur
    Test()
    {

    }
}
```

Main.java

```java
public class Main
{

    public static void main(String[] args)
    {
        // Instanciation d'un objet Test
        Test t = new Test();

        // Affichage des valeurs par défaut des attributs
        System.out.println("Le type int : " + t.x);
        System.out.println("Le type double : " + t.y);
        System.out.println("Le type booléen : " + t.b);

    }
}
```

```
Le type int : 0
Le type double : 0.0
Le type booléen : false
```

03

Retour sur le constructeur

03

Retour sur le constructeur

Le constructeur par défaut

- En réalité, en langage Java, un constructeur est attribué par défaut et implicitement à chaque classe.

- La syntaxe du constructeur par défaut est :

Il n'a pas de type de sortie et prend le nom de sa classe → **NomClasse** **()** ← Ne prend aucun paramètre en entrée

{ } ← Corps vide

- Ainsi, le code suivant est correct:

Point.java

```
public class Point
{
    // Liste des attributs
    int x;
    int y;
}
```

Main.java

```
public class Main
{
    public static void main(String[] args)
    {
        // Instanciation d'un objet Point
        Point p0 = new Point();
    }
}
```

Valide malgré l'absence d'un constructeur explicite

Retour sur le constructeur

Le constructeur par défaut

L'instanciation d'un objet en utilisant le constructeur par défaut permet :

- D'allouer l'espace mémoire à l'objet
- D'initialiser ses attributs automatiquement aux **valeurs par défaut** de leurs types respectifs

Comment faire si l'on souhaite créer un objet et initialiser ses attributs en même temps ?

2 solutions (au min)

1. Créer **une méthode** qui permettra d'initialiser les attributs d'un objet après sa création (son instanciation) ➔ **Mais je ne sais pas encore manipuler les méthodes**

2. **Définition explicite d'un constructeur avec paramètres**

Retour sur le constructeur

Le constructeur d'initialisation

- Le constructeur d'initialisation est un constructeur défini explicitement dans la classe
- Le constructeur d'initialisation peut avoir un ou plusieurs paramètres
- S'agissant d'un constructeur, il se présente comme étant une méthode particulière portant le nom de sa classe.

Point.java

```
public class Point
{
    // Liste des attributs
    int x;
    int y;
    // Constructeur d'initialisation
    Point(int abc, int ord)
    {
        x = abc;
        y = ord;
    }
}
```

Main.java

```
public class Main
{
    public static void main(String[] args)
    {
        // Instanciation d'un objet Point
        Point p0 = new Point(3, 6);
    }
}
```

Retour sur le constructeur

Le constructeur d'initialisation : Conflit

Observons le code suivant où :

- La classe ***Point*** contient un constructeur d'initialisation
- Le programme ***main*** crée deux instances **p0** (avec le constructeur par défaut) et **p1** (avec le constructeur d'initialisation)

?

```
1 related problem
public class Point
{
    // Liste des attributs
    int x;
    int y;
    // Constructeur d'initialisation
    1 related problem
    Point(int abc, int ord)
    {
        x = abc;
        y = ord;
    }
}
```

?

```
public class Main {
    public static void main(String[] args) {
        // Instanciation d'un objet Point avec un constructeur par défaut
        Point p0 = new Point();
        // Instanciation d'un objet Point avec un constructeur d'initialisation
        Point p1 = new Point( abc: 3,  ord: 6);
    }
}
```

?

Problème ? Pourquoi ?

Retour sur le constructeur

Le constructeur d'initialisation : Conflit

- Si aucun constructeur n'est spécifié, le compilateur génère automatiquement un constructeur par défaut qui initialise les attributs avec les valeurs par défaut.

- Dès qu'au moins un constructeur (différent) a été spécialisé, ce constructeur par défaut implicite **n'est plus valide**

➔ Si l'on spécifie un constructeur sans spécifier de constructeur par défaut, on ne peut plus construire d'objet de cette classe sans l'initialiser explicitement.

Retour sur le constructeur

Le constructeur d'initialisation : Solution au conflit

- Si l'on souhaite maintenir la création d'objets par le constructeur par défaut tout en ayant un constructeur

  ➔ l'ajouter à la classe **EXPLICITEMENT**

```java
public class Point
{
    // Liste des attributs
    int x;
    int y;
    // Constructeur d'initialisation
    Point(int abc, int ord)
    {
        x = abc;
        y = ord;
    }

    // Constructeur par défaut
explicite
    Point()
    {}
}
```

```java
public class Main
{
    public static void main(String[] args)
    {
        // Instanciation d'un objet Point avec un constructeur
par défaut
        Point p0 = new Point();
        // Instanciation d'un objet Point avec un constructeur
d'initialisation
        Point p1 = new Point(3, 6);
    }
}
```

Tout est bon

Retour sur le constructeur

La surcharge de constructeur

- **La surcharge d'une** méthode ou **d'un constructeur** permet de définir plusieurs fois une même méthode/constructeur avec des paramètres différents (type, nombre)
- Le compilateur choisit la méthode/constructeur qui doit être appelée en fonction du nombre et du type des arguments .

➔ **Il est possible d'avoir dans une même classe : un <u>constructeur par défaut</u>** et **plusieurs <u>constructeurs d'initialisation</u>** portant tous le même nom qui est le nom de leurs classes.

Retour sur le constructeur

La surcharge de constructeur

Exemple:

Point.java

```java
public class Point
{
    // Liste des attributs
    int x;
    int y;
    // Constructeur par défaut explicite
    Point()
    {}
    // Constructeur d'initialisation à deux paramètres
    Point(int abc, int ord)
    {
        x = abc;
        y = ord;
    }
    // Constructeur d'initialisation à un seul paramètre
    // L'autre attribut prendra la valeur par défaut
    Point(int abc)
    {
        x = abc;
    }
}
```

Main.java

```java
public class Main
{
    public static void main(String[] args)
    {
        // Instanciation d'un objet Point avec un
constructeur par défaut
        Point p0 = new Point();
        // Instanciation d'un objet Point avec un
        // constructeur d'initialisation à 2 paramètres
        Point p1 = new Point(3, 6);
        // Instanciation d'un objet Point avec un
        // constructeur d'initialisation à 1 paramètre
        Point p2 = new Point(12);
    }
}
```

Retour sur le constructeur

Modification des valeurs par défaut

- Java permet de donner directement une **valeur par défaut** aux attributs à travers ses constructeurs.
- Si le constructeur appelé ne modifie pas la valeur de cet attribut, ce dernier aura alors la valeur indiquée dans le corps du constructeur.

Exemple:

Point.java

```java
public class Point
{
    // Liste des attributs
    int x;
    int y;
    // Constructeur par défaut explicite
    Point()
    {
        x = 34;
        y = -12;
    }
    // Constructeur d'initialisation à un seul paramètre
    // L'autre attribut prendra la valeur par défaut
    Point(int abc)
    {
        x = abc;
        y = 3;
    } }
```

Main.java

```java
public class Main
{
    public static void main(String[] args)
    {
        // Instanciation d'un objet Point avec un
constructeur par défaut
        Point p0 = new Point();
        System.out.println("Constructeur par défaut :
");
        System.out.println("x = "+ p0.x+" y = "+p0.y);
        // Instanciation d'un objet Point avec un
        // constructeur d'initialisation à 1 paramètre
        Point p2 = new Point(12);
        System.out.println("Constructeur avec un
paramètre : ");
        System.out.println("x = "+ p2.x+" y = "+p2.y);
    }
}
```

```
Constructeur par défaut :
x = 34 y = -12
Constructeur avec un paramètre :
x = 12 y = 3
```

Retour sur le constructeur

Constructeur de copie

Exemple de motivation

Considérons les deux classes *Point* et *Main* suivantes où une instance **p0** est créé en utilisant un constructeur par défaut et affecté à un autre objet **p1**.

Point.java

```
public class Point
{
    // Liste des attributs
    int x;
    int y;
    // Constructeur par défaut explicite
    Point()
    {

    }
    // Constructeur d'initialisation à deux paramètres
    Point(int abc, int ord)
    {
        x = abc;
        y = ord;
    }
}
```

Main.java

```
public class Main
{
    public static void main(String[] args)
    {
        // Instanciation d'un objet Point avec un constructeur par défaut
        Point p0 = new Point();
        Point p1 = p0;
        System.out.println("x = "+ p1.x+" y = "+p1.y);
    }
}
```

Que va afficher ce programme comme valeur des attributs de p1 ?

Retour sur le constructeur

Constructeur de copie

Exemple de motivation

Considérons les deux classes *Point* et *Main* suivantes où une instance **p0** est créé en utilisant un constructeur par défaut et affecté à un autre objet **p1**.

Point.java

```
public class Point
{
    // Liste des attributs
    int x;
    int y;
    // Constructeur par défaut explicite
    Point()
    {

    }
    // Constructeur d'initialisation à deux paramètres
    Point(int abc, int ord)
    {
        x = abc;
        y = ord;
    }
}
```

Main.java

```
public class Main
{
    public static void main(String[] args)
    {
        // Instanciation d'un objet Point avec un constructeur par défaut
        Point p0 = new Point();
        Point p1 = p0;
        System.out.println("x = "+ p1.x+" y = "+p1.y);
    }
}
```

Réponse : x = 0 y = 0

Retour sur le constructeur

Constructeur de copie

Exemple de motivation

Essayons de modifier les valeurs des attributs de **p1** et afficher les valeurs des attributs de **p0**

Point.java

```
public class Point
{
    // Liste des attributs
    int x;
    int y;
    // Constructeur par défaut explicite
    Point()
    {

    }
    // Constructeur d'initialisation à deux paramètres
    Point(int abc, int ord)
    {
        x = abc;
        y = ord;
    }
}
```

Main.java

```
public class Main
{
    public static void main(String[] args)
    {
        // Instanciation d'un objet Point avec un constructeur par défaut
        Point p0 = new Point();
        Point p1 = p0;
        p1.x = 3;
        p1.y = 4;
        System.out.println("x = "+ p0.x+" y = "+p0.y);
    }
}
```

Que va afficher ce programme ?

Retour sur le constructeur

Constructeur de copie

Exemple de motivation

Essayons de modifier les valeurs des attributs de **p1** et afficher les valeurs des attributs de **p0**

Point.java

```
public class Point
{
    // Liste des attributs
    int x;
    int y;
    // Constructeur par défaut explicite
    Point()
    {

    }
    // Constructeur d'initialisation à deux paramètres
    Point(int abc, int ord)
    {
        x = abc;
        y = ord;
    }
}
```

Main.java

```
public class Main
{
    public static void main(String[] args)
    {
        // Instanciation d'un objet Point avec un constructeur par défaut
        Point p0 = new Point();
        Point p1 = p0;
        p1.x = 3;
        p1.y = 4;
        System.out.println("x = "+ p0.x+" y = "+p0.y);
    }
}
```

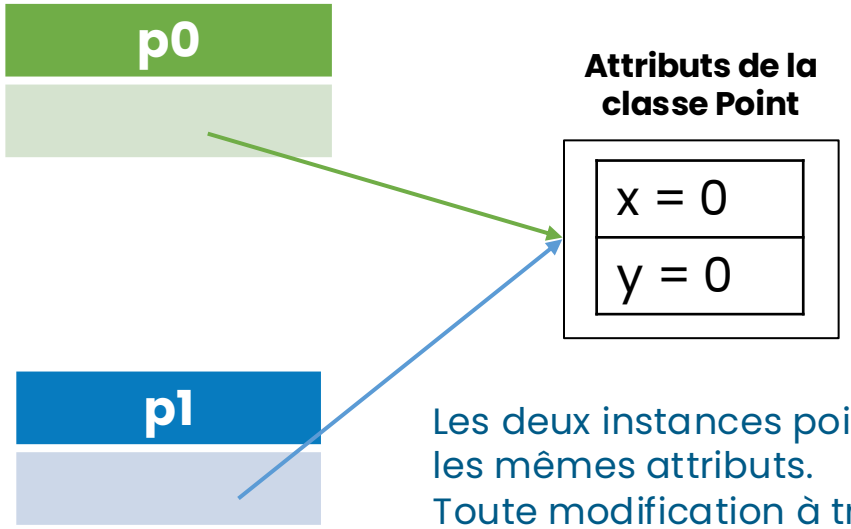
Réponse : x = 3 y = 4

Pourquoi ???!!!

Retour sur le constructeur

Constructeur de copie

Que se passe-t-il lors de l'affectation d'un objet à un autre ?



Les deux instances pointent vers les mêmes attributs.
Toute modification à travers p0 de x et y est visible par p1 et vice versa

Main.java

```
public class Main
{
    public static void main(String[] args)
    {
        // Instanciation d'un objet Point avec un constructeur par défaut
        Point p0 = new Point();
        Point p1 = p0;
        p1.x = 3;
        p1.y = 4;
        System.out.println("x = "+ p0.x+" y = "+p0.y);
    }
}
```

Comment faire pour les séparer ? → **Constructeur de copie**

Retour sur le constructeur

Constructeur de copie

Le constructeur de copie permet d'initialiser une instance **en copiant** les attributs d'une autre instance de même type.

Comment ?

En prenant en paramètre un objet du même type de sa classe afin de créer sa copie.

Syntaxe

Type du paramètre qui est du même type de la classe courante

Nom du constructeur qui est le même que le nom de sa classe

```
NomClasse (NomClasse autre)
{
        attribut1 = autre.attribut1 ;
        ....
}
```

Copie des attributs un par un

Retour sur le constructeur

Constructeur de copie

Exemple

```java
public class Point
{
    // Liste des attributs
    int x;
    int y;
    // Constructeur par défaut explicite
    Point()
    {

    }
    // Constructeur de copie
    Point(Point o)
    {
        x = o.x;
        y = o.y;
    }
    // Constructeur d'initialisation à deux paramètres
    Point(int abc, int ord)
    {
        x = abc;
        y = ord;
    }
}
```

```java
public class Main
{
    public static void main(String[] args)
    {
        // Instanciation d'un objet Point avec un constructeur par
défaut
        Point p0 = new Point();
        System.out.println("Info P0 : x = "+ p0.x+" y = "+p0.y);

        Point p1 = new Point(p0);

        p1.x = 3;
        p1.y = 4;
        System.out.println("Info P1 : x = "+ p1.x+" y = "+p1.y);

    }
}
```

```
Info P0 : x = 0 y = 0
Info P1 : x = 3 y = 4
```

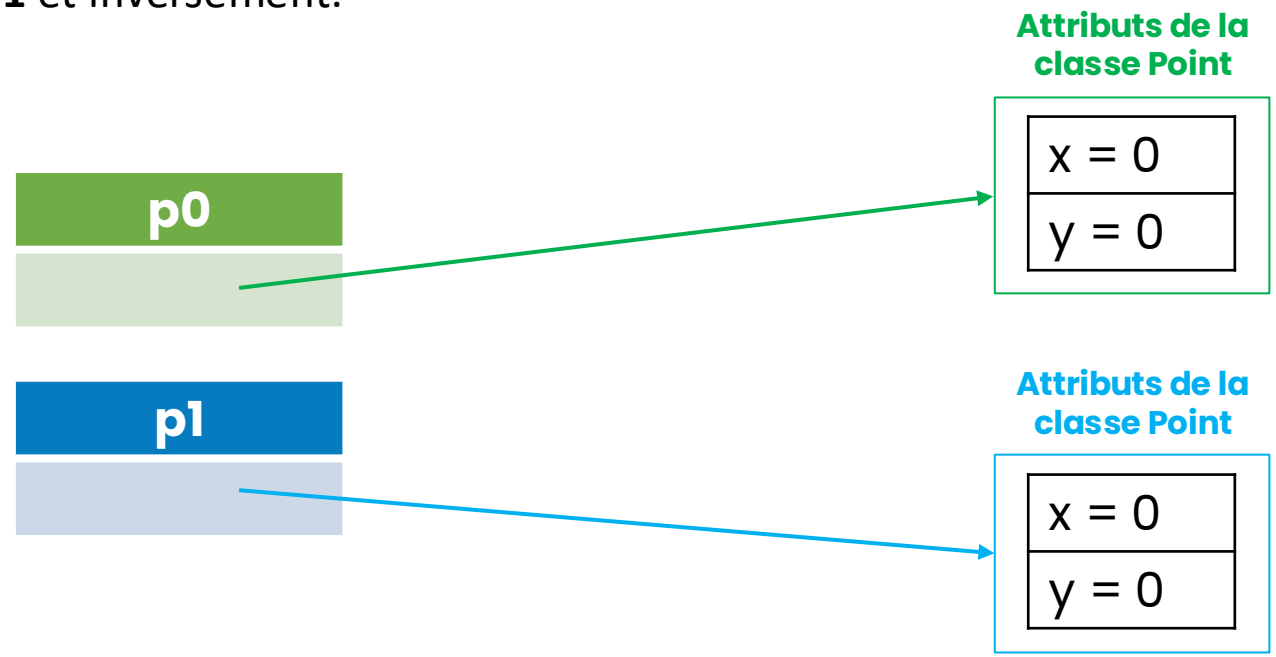
Retour sur le constructeur

Constructeur de copie

Explication

La construction de **p1** en utilisant un constructeur de copie et en passant par « new » permet de créer une nouvelle référence complètement indépendante de **p0**.

Ainsi, initialement les valeurs des attributs de **p1** sont les mêmes que **p0** , mais les modifications effectuées sur **p0** n'impactent pas **p1** et inversement.



Retour sur le constructeur

Le mot clé « this »

- Le mot-clé ***this*** est utilisé dans une classe pour désigner l'instance courante de la classe elle-même. Il est utilisé à différentes fins.

  1. **Rendre le code explicite et non ambigu**

     Si dans une méthode/constructeur, il y a un paramètre ayant le même nom qu'un attribut de la classe dont la méthode/le constructeur fait partie, il est possible de désigner explicitement l'attribut grâce à ***this***

```
public class Point
{
    // Liste des attributs
    int x;
    int y;

    // Constructeur d'initialisation à deux paramètres
    Point(int x, int y)
    {
        this.x = x;
        this.y = y;
    }
}
```

Représentent les attributs de la classe

Représentent les paramètres du constructeur

Retour sur le constructeur

Le mot clé « this »

- Le mot-clé ***this*** est utilisé dans une classe pour désigner l'instance courante de la classe elle-même. Il est utilisé à différentes fins.

2. Appeler un autre constructeur de la classe

Un constructeur peut appeler un autre constructeur de la classe en utilisant le mot-clé **this** comme nom appelé.

```
public class Point
{
    // Liste des attributs
    int x;
    int y;

    Point(Point o)
    {
        this(0, 0);
    }
    Point(int x, int y)
    {
        this.x = x;
        this.y = y;
    }
}
```

Appel du constructeur à deux paramètres

04

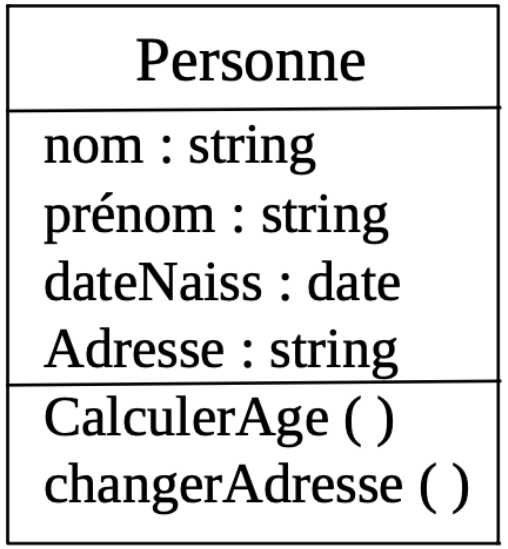
Méthodes de classe

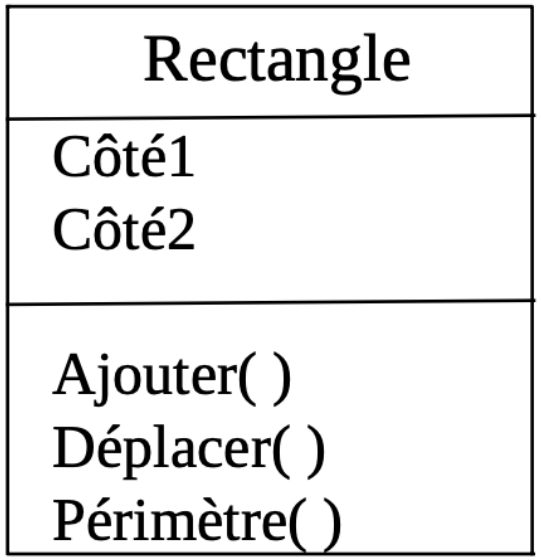
04

Les méthodes de classe

Définition

- Une méthode est la mise en œuvre d'une opération pour une Classe (plusieurs méthodes pour le même service)
- Une méthode définit une fonction applicable aux objets de la classe





Les méthodes de classe

Déclaration d'une méthode

Une méthode se déclare comme une fonction dans une classe.

Syntaxe :

```
TypeDeSortie  nomMethode (Liste des paramètres …)
{
    // Corps de la méthode
}
```

Les méthodes de classe

Appel d'une méthode

Pour appeler une méthode de classe, il est obligatoire de passer par une instance d'objet de la même classe.

Syntaxe :

nomObjet.nomMethode (**Liste des arguments …**)

ou

variable = **nomObjet.nomMethode** (**Liste des arguments …**)

Les méthodes de classe

Appel d'une méthode

Exemple :

```java
public class Point
{
    // Liste des attributs
    int x;
    int y;
    // Constructeur par défaut explicite
    Point()
    {
    }
    // Constructeur d'initialisation à deux paramètres
    Point(int abc, int ord)
    {
        x = abc;
        y = ord;
    }
    // Méthode de classe
    public void afficher()
    {
        System.out.println("Info Point : x = "+ this.x+" y = "+this.y);
    }
}
```

```java
public class Main
{
    public static void main(String[] args)
    {
        Point p0 = new Point();
        p0.afficher();

        Point p1 = new Point(3, 4);
        p1.afficher();
    }
}
```

```
Info Point : x = 0 y = 0
Info Point : x = 3 y = 4
```

Activité

Faire l'exercice 1 du TD1.